Ingresso Campus — Product Requirements Document

Version: 1.0 **Date:** 2026-10-16 **Author:** PRD Generator **Status:** Draft

> *"Ingresso Campus" é um nome provisório. Premissas assumidas: web app em Next.js + PostgreSQL, pagamentos via Pagar.me (Pix e cartão) com split, MVP construído por uma pessoa com ajuda de IA, mercado inicial em uma capital do Nordeste.*

Executive Summary

Ingresso Campus é uma plataforma de venda de ingressos e inscrições para eventos universitários e acadêmicos (festas de atléticas, semanas acadêmicas, congressos). Atende organizadores (atléticas, centros acadêmicos, comissões) e estudantes compradores. Resolve o problema de plataformas genéricas com taxas percentuais opacas, repasses lentos e pouca ajuda com certificados. O resultado esperado é que organizadores vendam ingressos por Pix e cartão em minutos, façam check-in por QR code e recebam o repasse em até 2 dias após o evento.

Problem Statement

Current state: organizadores universitários vendem por plataformas genéricas, por Pix manual com planilha ou por listas no WhatsApp. Certificados são feitos à mão.

Pain points:

1. Taxas percentuais altas e pouco claras para o comprador.
2. Repasse demorado e sem previsibilidade.
3. Conferência manual de pagamentos Pix, com fraude e duplicidade.
4. Certificados emitidos manualmente após o evento.
5. Suporte impessoal quando algo dá errado.

Impact: perda de tempo da comissão organizadora, ingressos falsos ou duplicados e menos vendas por checkout ruim.

Goals & Success Metrics

Goal	Metric	Target	Measurement Method
Validar o produto	Eventos concluídos com parceiros	5 em 90 dias	Banco de dados
Checkout rápido	Taxa de conclusão do checkout	> 60%	Analytics
Repasse confiável	Repasses pagos em até D+2	100%	Painel financeiro
Segurança	Taxa de estorno	< 1% do GMV	Gateway
Retenção	Organizadores que repetem evento	> 30% em 6 meses	Banco de dados

User Personas

Organizador (presidente ou tesoureiro de atlética)

- **Role:** estudante que organiza festas e eventos
- **Goals:** vender ingressos rápido, controlar entrada, receber o dinheiro sem dor de cabeça
- **Pain points:** planilhas, golpes com comprovantes de Pix, repasse demorado
- **Technical proficiency:** Média
- **Usage context:** celular, em semanas de venda e no dia do evento

Comissão de congresso

- **Role:** professor ou estudante que coordena inscrição e certificados de um congresso
- **Goals:** inscrições, pagamento, lista de presença e certificados automáticos
- **Pain points:** processo manual e trabalho repetitivo
- **Technical proficiency:** Média
- **Usage context:** desktop, durante o planejamento, e celular no evento

Comprador (estudante)

- **Role:** universitário de 18 a 30 anos
- **Goals:** comprar o ingresso em poucos toques e recebê-lo no celular
- **Pain points:** taxas surpresa, checkout longo, ingresso que não chega
- **Technical proficiency:** Alta no celular
- **Usage context:** mobile, vindo de Instagram e WhatsApp

Functional Requirements

FR-001: Cadastro e login

Description: usuários criam conta com e-mail e senha ou login social. **User story:** As an organizador, I want to create an account quickly so that I start selling today. **Acceptance criteria:**

- [ ] Cadastro com e-mail e senha, e login com Google
- [ ] Verificação de e-mail antes de publicar um evento
- [ ] Senha com no mínimo 8 caracteres **Priority:** P0 **Dependencies:** nenhuma

FR-002: Criar e publicar evento

Description: o organizador cria o evento com nome, descrição, data, local, imagem e URL pública. **User story:** As an organizador, I want to publish an event page so that people can buy tickets. **Acceptance criteria:**

- [ ] Campos obrigatórios: nome, data e hora, local, descrição
- [ ] URL pública no formato `/e/{slug}` e slug único
- [ ] Evento pode ser salvo como rascunho e publicado depois
- [ ] Página pública carrega em menos de 2 s em 4G **Priority:** P0 **Dependencies:** FR-001

FR-003: Tipos e lotes de ingresso

Description: o evento tem tipos de ingresso (por exemplo, "Pista") e lotes com preço, quantidade e período. **User story:** As an organizador, I want to create ticket lots so that I control price and stock. **Acceptance criteria:**

- [ ] Preço em centavos (inteiro), quantidade total e período de venda por lote

- [ ] Estoque nunca fica negativo, mesmo com compras simultâneas
- [ ] Ingresso gratuito permitido (preço 0)
- [ ] Meia-entrada como tipo opcional com regra de comprovação na entrada **Priority:** P0 **Dependencies:** FR-002

FR-004: Checkout com Pix e cartão

Description: o comprador paga com Pix ou cartão de crédito sem criar conta (apenas nome, e-mail, CPF e telefone). **User story:** As a comprador, I want to pay by Pix in few taps so that I buy before the lot ends. **Acceptance criteria:**

- [ ] Pix gera QR code e copia-e-cola, com expiração de 15 minutos
- [ ] Cartão com até 3 parcelas, sem armazenar dados do cartão no servidor
- [ ] Resumo mostra preço do ingresso, taxa de serviço e total antes de pagar
- [ ] Reserva do estoque por 15 minutos durante o pagamento
- [ ] Pagamento confirmado por webhook em até 30 s **Priority:** P0 **Dependencies:** FR-003, integração Pagar.me

FR-005: Emissão de ingresso com QR code

Description: após a confirmação do pagamento, o comprador recebe o ingresso com QR code único. **User story:** As a comprador, I want to get my ticket instantly so that I can enter the event. **Acceptance criteria:**

- [ ] Ingresso enviado por e-mail em até 60 s da confirmação
- [ ] Ingresso disponível em página `/ingresso/{token}` sem login
- [ ] QR code com token assinado, impossível de adivinhar
- [ ] Reenvio de ingresso pelo e-mail ou pelo organizador **Priority:** P0 **Dependencies:** FR-004

FR-006: Check-in por QR code

Description: a equipe lê o QR code pelo celular no dia do evento. **User story:** As an organizador, I want to scan tickets at the door so that no duplicate enters. **Acceptance criteria:**

- [ ] Leitura pela câmera no navegador (PWA), com resultado em menos de 1 s
- [ ] Ingresso já usado mostra aviso em vermelho com horário do uso anterior
- [ ] Busca manual por nome ou CPF quando o QR falha
- [ ] Funciona com lista offline pré-carregada (sincroniza ao reconectar) **Priority:** P0 **Dependencies:** FR-005

FR-007: Painel do organizador

Description: vendas, receita, lista de compradores e exportação em CSV. **User story:** As an organizador, I want to see sales in real time so that I make decisions. **Acceptance criteria:**

- [ ] Mostra ingressos vendidos, receita líquida e taxa por lote
- [ ] Exporta lista de compradores em CSV
- [ ] Atualização em até 1 minuto após cada venda **Priority:** P0 **Dependencies:** FR-004

FR-008: Repasse ao organizador

Description: o gateway divide o pagamento (split) e o organizador recebe após o evento. **User story:** As an organizador, I want to receive my money fast so that I pay suppliers. **Acceptance criteria:**

- [ ] Cadastro de conta bancária ou chave Pix com validação do titular (CPF/CNPJ)
- [ ] Repasse liberado em D+3 após o evento no plano Essencial e D+2 nos planos pagos
- [ ] Extrato com cada venda, taxa e valor líquido **Priority:** P1 **Dependencies:** FR-004

FR-009: Reembolso e cancelamento

Description: o comprador pede reembolso, e o organizador reembolsa ou cancela o evento. **User story:** As a comprador, I want to get a refund within the legal period so that I am protected. **Acceptance criteria:**

- [ ] Reembolso integral, inclusive da taxa de serviço, quando o evento é cancelado
- [ ] Pedido de arrependimento em até 7 dias da compra, quando aplicável pela política do evento
- [ ] Ingresso reembolsado fica inválido imediatamente **Priority:** P1 **Dependencies:** FR-004

FR-010: Certificados automáticos

Description: emissão de certificados em PDF para participantes com check-in. **User story:** As a comissão, I want to issue certificates automatically so that I save days of work. **Acceptance criteria:**

- [ ] Modelo de certificado com nome, evento, carga horária e assinatura
- [ ] Emissão em lote para todos os participantes com check-in
- [ ] Certificado com código de validação em página pública **Priority:** P1 **Dependencies:** FR-006

FR-011: Cupons de desconto

Description: cupons por percentual ou valor, com limite de uso. **User story:** As an organizador, I want to create coupons so that I reward members and partners. **Acceptance criteria:**

- [ ] Cupom com código, valor, validade e limite de usos
- [ ] Desconto nunca deixa o total abaixo de zero **Priority:** P1 **Dependencies:** FR-004

FR-012: Notificações por WhatsApp

Description: confirmação de compra e lembrete do evento por WhatsApp. **User story:** As a comprador, I want to receive my ticket on WhatsApp so that I find it easily. **Acceptance criteria:**

- [ ] Mensagem de confirmação em até 2 minutos
- [ ] Lembrete 24 h antes do evento **Priority:** P2 **Dependencies:** FR-005

FR-013: Submissão de trabalhos (congressos)

Description: autores enviam resumos e a comissão avalia. **User story:** As a comissão, I want to receive and review submissions so that I build the program. **Acceptance criteria:**

- [ ] Formulário com título, autores, resumo e arquivo PDF
- [ ] Status aprovado, reprovado ou pendente por trabalho **Priority:** P2 **Dependencies:** FR-002

FR-014: Planos de assinatura do organizador

Description: planos Essencial, Pro e Congresso com taxas e recursos diferentes. **User story:** As an organizador recorrente, I want to subscribe so that I pay less per ticket. **Acceptance criteria:**

- [ ] Taxa por ingresso calculada pela faixa de preço e pelo plano
- [ ] Assinatura mensal ou anual via Pagar.me ou Stripe **Priority:** P2 **Dependencies:** FR-008

Non-Functional Requirements

Performance

- Página de evento: carrega em menos de 2 s em 4G
- API de checkout: resposta abaixo de 500 ms (p95)
- Concorrência: suportar 200 compras simultâneas no mesmo evento

Security

- Autenticação: sessão com cookie HttpOnly (Auth.js)
- Dados de cartão: nunca tocam o servidor (tokenização do gateway)
- Dados em trânsito (HTTPS) e em repouso (criptografia do banco gerenciado)
- Conformidade: LGPD (consentimento, exclusão de dados sob pedido, política de privacidade)
- CPF armazenado criptografado

Scalability

- Carga inicial: até 500 ingressos por evento, 10 eventos simultâneos
- Meta em 12 meses: 50 mil ingressos por ano, picos de 100 compras por minuto

Availability

- Uptime alvo: 99,5%
- Backup diário do banco, com restauração testada
- RPO de 24 h, RTO de 4 h

Technical Architecture

System Overview

Aplicação web monolítica em Next.js com API Routes, banco PostgreSQL e uma fila para tarefas assíncronas (e-mails, certificados, webhooks).

Technology Stack

- **Frontend:** Next.js (React, TypeScript), Tailwind CSS, PWA para check-in
- **Backend:** Next.js API Routes (TypeScript)
- **Database:** PostgreSQL gerenciado (Neon ou Supabase), com Prisma
- **Hosting:** Vercel para a aplicação e um worker em Railway para a fila
- **Key libraries:** Prisma, Zod, Auth.js, React Hook Form, `qrcode`, `@react-pdf/renderer`, BullMQ com Redis (ou pg-boss), Resend para e-mail

Architecture Diagram (Description)

O navegador do comprador acessa a página do evento em Next.js. O checkout chama a API, que reserva o estoque no PostgreSQL dentro de uma transação e cria a cobrança no Pagar.me (Pix ou cartão) com split para o organizador. O Pagar.me envia webhook de confirmação para a API, que marca o pedido como pago, gera os ingressos e enfileira o e-mail com o QR code. O app de check-in (PWA) consulta a API para validar tokens e marca o ingresso como usado.

API Specifications

POST /api/checkout

Purpose: cria um pedido, reserva estoque e gera a cobrança. **Authentication:** None

Request:

```
{
  "eventId": "uuid — evento",
  "items": [{ "ticketTypeId": "uuid", "quantity": "int — 1 a 6" }],
  "buyer": { "name": "string", "email": "string", "cpf": "string", "phone": "string"
},
  "paymentMethod": "pix | card",
  "cardToken": "string — obrigatório se card",
  "couponCode": "string — opcional"
}
```

Response (200):

```
{
  "orderId": "uuid",
  "status": "pending",
  "total": "int — centavos, com taxa de serviço",
  "pix": { "qrCode": "string — base64", "copyPaste": "string", "expiresAt": "ISO
date" }
}
```

Error responses:

- 400: dados inválidos
- 409: estoque insuficiente
- 422: cupom inválido ou expirado

POST /api/webhooks/pagarme

Purpose: recebe a confirmação de pagamento. **Authentication:** assinatura HMAC do gateway

Request:

```
{ "type": "string — order.paid", "data": { "id": "string", "status": "string" } }
```

Response (200):

```
{ "ok": true }
```

Error responses:

- 401: assinatura inválida
- 404: pedido não encontrado (responder 200 após registrar, para não gerar reentrega infinita)

POST /api/checkin

Purpose: valida e marca um ingresso como usado. **Authentication:** Required (equipe do evento)

Request:

```
{ "token": "string — conteúdo do QR code", "eventId": "uuid" }
```

Response (200):

```
{ "result": "valid | already_used | invalid", "holderName": "string", "usedAt": "ISO date | null" }
```

Error responses:

- 401: sem permissão
- 404: ingresso de outro evento

UI/UX Requirements

Página pública do evento

Purpose: convencer e vender. **Key elements:**

- Imagem, nome, data, local e descrição
- Seletor de ingressos com preço final (ingresso + taxa)
- Botão "Comprar" fixo no rodapé no celular

User flow:

1. Estudante abre o link do Instagram
2. Escolhe o ingresso e a quantidade
3. Preenche nome, e-mail, CPF e telefone, e paga por Pix
4. Vê o ingresso com QR code

States:

- Empty state: "Vendas ainda não começaram" com data do primeiro lote
- Loading state: esqueleto da página
- Error state: mensagem clara e botão para tentar de novo; "Esgotado" quando acabar o estoque

Painel do organizador

Purpose: criar eventos e acompanhar vendas. **Key elements:**

- Cartões de vendas, receita líquida e check-ins
- Tabela de pedidos com filtro e exportação CSV
- Botão "Abrir check-in"

User flow:

1. Organizador cria o evento em um formulário de 3 passos
2. Cria lotes e publica
3. Compartilha o link
4. Acompanha as vendas

States:

- Empty state: guia "Crie seu primeiro evento"
- Loading state: esqueleto
- Error state: alerta com botão de tentar de novo

App de check-in (PWA)

Purpose: validar ingressos na porta. **Key elements:** câmera, resultado em tela cheia (verde, vermelho, amarelo), contador de entradas, busca manual **User flow:** abrir, apontar a câmera, ver o resultado em menos de 1 s **States:** modo offline com aviso; câmera negada mostra busca manual

Data Models

User

Field	Type	Required	Description
id	UUID	Yes	Chave primária
email	string	Yes	Único
name	string	Yes	Nome
password_hash	string	No	Vazio no login social
role	enum(organizer, admin)	Yes	Papel
created_at	timestamp	Yes	Criação

Event

Field	Type	Required	Description
id	UUID	Yes	Chave primária
organizer_id	UUID	Yes	FK para User
slug	string	Yes	Único
title	string	Yes	Nome
description	text	Yes	Descrição
starts_at	timestamp	Yes	Início
venue	string	Yes	Local
status	enum(draft, published, cancelled, finished)	Yes	Estado
fee_mode	enum(buyer, organizer)	Yes	Quem paga a taxa

TicketType

Field	Type	Required	Description
id	UUID	Yes	Chave primária
event_id	UUID	Yes	FK para Event
name	string	Yes	Nome do lote
price_cents	int	Yes	Preço em centavos
quantity	int	Yes	Estoque total
sold	int	Yes	Quantidade vendida
sales_start	timestamp	Yes	Início das vendas

sales_end	timestamp	Yes	Fim das vendas

Order

Field	Type	Required	Description
id	UUID	Yes	Chave primária
event_id	UUID	Yes	FK para Event
buyer_name / buyer_email	string	Yes	Dados do comprador
buyer_cpf	string (criptografado)	Yes	CPF
subtotal_cents / fee_cents / total_cents	int	Yes	Valores
status	enum(pending, paid, expired, refunded)	Yes	Estado
gateway_id	string	No	ID no Pagar.me
expires_at	timestamp	Yes	Fim da reserva

Ticket

Field	Type	Required	Description
id	UUID	Yes	Chave primária
order_id	UUID	Yes	FK para Order
ticket_type_id	UUID	Yes	FK para TicketType
token	string	Yes	Token assinado, único
holder_name	string	Yes	Nome
checked_in_at	timestamp	No	Horário do check-in

Relationships: User 1:N Event; Event 1:N TicketType; Event 1:N Order; Order 1:N Ticket.

Indexes: `Event.slug` (único), `Ticket.token` (único), `Order(event_id, status)`, `Order.gateway_id`.

Integration Points

Pagar.me

Purpose: cobrança Pix e cartão com split **Integration type:** REST API e webhook **Data exchanged:**

- Inbound: confirmações de pagamento, estornos
- Outbound: criação de pedido, dados do comprador, regras de split **Authentication:** chave secreta no servidor, webhook com HMAC **Rate limits:** conforme o contrato **Fallback behavior:** se a criação da

cobrança falhar, o pedido expira e o estoque é liberado; webhooks são reprocessados por job de conciliação a cada 5 min

Resend (e-mail)

Purpose: envio de ingressos e certificados **Integration type:** REST API **Fallback behavior:** o ingresso fica sempre disponível em `/ingresso/{token}`

Edge Cases & Error Handling

Scenario	Expected Behavior
Duas pessoas compram o último ingresso	A transação garante só um sucesso; a outra recebe "esgotado"
Pix pago após a expiração	Conciliação reembolsa automaticamente ou emite o ingresso se houver estoque
Webhook duplicado	Idempotência por `gateway_id`; não gera ingressos duplicados
QR code lido duas vezes	Segunda leitura mostra "já usado" com horário
Evento cancelado	Todos os pedidos pagos são reembolsados, com a taxa
Comprador digita e-mail errado	Ingresso acessível pelo link da página de confirmação

Error Handling Strategy

- **User-facing errors:** mensagem em português, sem códigos técnicos
- **System errors:** log estruturado e alerta (Sentry) para falhas de pagamento e webhook
- **Retry logic:** envio de e-mail com até 5 tentativas com backoff exponencial
- **Graceful degradation:** se o e-mail falhar, o ingresso continua na página pública

Testing Requirements

Unit Tests

- Cálculo de taxa por faixa e plano
- Cálculo de total com cupom
- Geração e verificação do token do QR code

Integration Tests

- Checkout completo com gateway em sandbox
- Webhook idempotente

E2E Tests

- Organizador cria evento e publica; comprador paga por Pix; ingresso é validado no check-in

Performance Tests

- 200 compras simultâneas do mesmo lote sem vender além do estoque

Implementation Notes for AI

Build Order

1. Modelos de dados no Prisma e migrações

2. Autenticação e cadastro de organizador
3. CRUD de evento e lotes
4. Página pública do evento
5. Checkout com reserva de estoque e integração Pagar.me (sandbox)
6. Webhook, emissão de ingresso e e-mail
7. Painel do organizador
8. PWA de check-in
9. Repasse e reembolso
10. Certificados, cupons, assinaturas

File Structure Suggestion

```
/src
  /app
    /e/[slug]
    /ingresso/[token]
    /painel
    /checkin
    /api
  /lib
    /fees.ts
    /payments
    /tickets
  /server
    /jobs
  /types
/prisma
```

Critical Implementation Details

- **Dinheiro:** sempre em centavos inteiros, nunca em ponto flutuante.
- **Estoque:** reservar com `UPDATE ... WHERE sold + qty <= quantity` dentro de uma transação, nunca em duas etapas.
- **Idempotência:** webhooks e criação de cobrança usam chave idempotente.
- **Tokens do QR:** HMAC assinado, sem dados pessoais no QR.
- **Taxa:** calcular no servidor, nunca confiar em valor enviado pelo cliente.
- **Datas:** salvar em UTC e exibir no fuso de Fortaleza ou Recife.
- **LGPD:** CPF criptografado e rota de exclusão de dados.

Code Style Preferences

- TypeScript estrito, validação com Zod nas bordas
- Nomes de arquivos em kebab-case, componentes em PascalCase
- Lógica de negócio fora dos componentes, em `/lib`

Libraries to Use

- Prisma, pelo tipo seguro e migrações
- Zod, para validar entradas
- Auth.js, para sessão e login social
- Resend, pelo envio de e-mail simples

Libraries to Avoid

- Armazenar dados de cartão ou usar formulários próprios de cartão (usar a tokenização do gateway)

Common Pitfalls

- Vender além do estoque: usar atualização atômica no banco.
- Perder webhooks: criar job de conciliação.
- Fraude com cartão: ativar antifraude do gateway e limitar o cartão em lotes caros.

Testing Approach

- Testar com prioridade o checkout, o cálculo de taxas e o check-in
- Pular testes de telas puramente visuais
- Usar Vitest para unidade e Playwright para E2E